EECS 6320 - Final Project Action Plan - Group 1

Dataset: HMDA Modified Loan/Application Register (LAR). The dataset contains 13,000 records with 85 columns from the year 2023. Additional years will be added if time permits.

Problem Definition: Morgan Stanley private bank dataset contains important information into the procedure and acceptance criteria for loans and mortgages. Our goal is to find biases that exist in the data and correct them using multiple debiasing strategies. In the end we hope to compare the debiasing strategies for action taken to see which strategy works best for this dataset.

Methods: The methods we want to apply for debiasing are oversampling & weighted BCE loss, MMD distribution alignment, adversarial debiasing, and threshold selection. These strategies should allow for flexibility and variety to debias the dataset.

Potential Formulation: We have added the formulas of each of the methods on the next page.

Experiment Design: The two main sensitive attributes for this dataset are race and sex. For race, we will limit ourselves to 3-5 races out of the list for the protected groups. The secondary sensitive attributes we will look at are ethnicity, income, and age. The target/dependent variable that we are interested in for this experiment is action taken. The experiment will be to see the action taken distribution for each sensitive attribute and protected group and then to see which debiasing methods work best to remove bias from the dataset.

Look into intersection between race and debt to income ratio.

Fairness Constraints: Group Disparity Difference, Equal Opportunity, False Positive Rate gap and Individual Parity.

Responsibilities of Team Members:

- Abhinav will write the action plan and code the oversampling and weighted BCE loss.
- Joseph will preprocess the dataset, train the model using random forest and find the baseline bias.
- Mahin will code the adversarial debiasing method.
- Mohsin will code the threshold selection method.
- Asad will code the MMD distribution alignment method.

Any remaining tasks will be split equally between the team members such as presentation slides, debugging, or finalizing the code.

Timeline: The timeline for our project will be to obtain the dataset, preprocess the dataset, find the baseline bias, perform debiasing methods, compare results, and add secondary attributes if time permits.

Milestones:

- By the Q1 deliverable on March 6th the goal is to find the dataset, define the problem, and create an action plan.
- By the Q2 deliverable on March 13th the goal is to preprocess the dataset and find the baseline bias.
- By the Q3 deliverable on March 27th the debiasing methods will be coded and tested on the main attributes. The secondary attributes will be tested as will if time permits.
- By the Q4 deliverable on April 6th the presentation slides will be created and code will be finalized.

Fairness in Loans and Mortgages

- HMDA Modified Loan/Application Register (LAR), 2024 and backwards available (US)
  https://ffiec.cfpb.gov/data-publication/modified-lar/2023

  I downloaded 2023 Morgan Stanley data header included
  https://drive.google.com/file/d/1wlc6cKJqyqo-r5TKtmRklaO29M2c7RKC/view?usp=sharing

  SCHEMA (what columns mean)
  https://ffiec.cfpb.gov/documentation/publications/modified-lar/modified-lar-schema
  https://files.ffiec.cfpb.gov/documentation/2018-public-LAR-code-sheet.pdf

2- Potential formulation (idea + formula?)

- MMD formula:

Most common kernel used in MMD: RBF (Gaussian kernel)

$$k(x, y) = \exp\left(-\frac{\|x - y\|^2}{2\sigma^2}\right)$$

Where:

- x, y = feature vectors
- $\|x - y\|^2$ = squared Euclidean distance
- σ = bandwidth parameter

Interpretation

Distance between points	Kernel value
very small	close to **1**
medium	between **0 and 1**
very large	close to **0**

So the kernel converts **distance → similarity**.

$$\mathrm{MMD}^2(Z_m, Z_f) = \frac{1}{n_m^2}\sum_{i=1}^{n_m}\sum_{j=1}^{n_m} k(z_i^m, z_j^m) + \frac{1}{n_f^2}\sum_{i=1}^{n_f}\sum_{j=1}^{n_f} k(z_i^f, z_j^f) - \frac{2}{n_m n_f}\sum_{i=1}^{n_m}\sum_{j=1}^{n_f} k(z_i^m, z_j^f)$$

Where:

- Z_m = feature representations for group m
- Z_f = feature representations for group f
- n_m = number of samples in group m
- n_f = number of samples in group f
- $k(\cdot, \cdot)$ = kernel function (e.g., RBF kernel)
- z_i^m = the i-th sample from group m
- z_j^f = the j-th sample from group f

- Oversampling and weighted BCE loss
  - Oversampling:

Oversampling Formulation (Generalized)

$$n_{\text{needed}} = n_{g_1,\, y=1} - n_{g_0,\, y=1}$$

Where:

- g = sensitive **group**
- y = label
- $y = 1$ = positive outcome (e.g., income $> 50K$)
- $n_{g,y}$ = number of samples belonging to **group** g with **label** y

Meaning

- $n_{g_1,1}$ = number of **positive samples in group** g_1
- $n_{g_0,1}$ = number of **positive samples in group** g_0
- n_{needed} = number of **additional samples required to balance the groups**

Goal of Oversampling

$$n_{g_0,1}^{new} = n_{g_1,1}$$

Meaning the **positive outcomes for both groups become equal**.

  - Weighted BCE loss:

Weight Function

$$w(g, l) = \frac{P(g)P(l)}{P(g, l)}$$

Variables

- $w(g, l)$ — Weight assigned to samples belonging to **group** g with **label** l
- g — Sensitive **group membership** (e.g., demographic group)
- l — **Label/class value** (for binary classification $l \in \{0, 1\}$)
- $P(g)$ — Probability of samples belonging to group g in the training dataset
- $P(l)$ — Probability of label l in the training dataset
- $P(g, l)$ — Joint probability of samples belonging to **group** g and having **label** l

This weight increases the influence of **underrepresented group-label pairs**.

Weighted Binary Cross-Entropy Loss

$$L = \frac{1}{N} \sum_{i=1}^{N} w_i \left[-y_i \log(\hat{y}_i) - (1 - y_i) \log(1 - \hat{y}_i) \right]$$

Variables

- L — Final **training loss** minimized by the model
- N — Total number of **training samples**
- i — Index of a training sample
- w_i — Weight assigned to sample i based on its group and label
- y_i — **True label** of sample i (0 or 1)
- $\hat{y}_i$ — **Predicted probability** that sample i belongs to the positive class
- $\log$ — Natural logarithm

- Threshold selection

$$\text{cost}(t) = FP(t) + c\,FN(t)$$

$$t^* = \frac{1}{1 + c}$$

Where c represents the relative cost of false negatives compared to false positives.

- Adversarial debiasing

Model Components

Let $x \in \mathbb{R}^d$ be the input feature vector.

Where:

- x : input features of a data sample
- $\mathbb{R}^d$: d-dimensional real-valued feature space
- d : number of input features

Feature Representation

The predictor first learns a hidden representation:

$$z = h_\theta(x)$$

Symbols

- z : hidden representation (latent features)
- $h_\theta(\cdot)$: feature extractor network
- θ : model parameters (weights of the feature extractor)

Prediction Model

The predictor produces the final prediction:

$$\hat{y} = f_\theta(z)$$

Symbols

- $\hat{y}$ (y-hat): predicted probability of the target
- y: true label
- $f_\theta(\cdot)$: prediction model (classifier)
- θ: model parameters (weights of the predictor)

Prediction Loss

We use Binary Cross Entropy (BCE).

$$L_y = -\left[y \log(\hat{y}) + (1-y)\log(1-\hat{y})\right]$$

Symbols

- L_y : prediction loss
- y : true label
- $\hat{y}$ (y-hat) : predicted probability

Adversary Network

We introduce an adversary that tries to predict the protected attribute.

$$\hat{a} = g_\phi(z)$$

Symbols

- a : protected attribute (e.g., gender or race)
- $\hat{a}$ (a-hat) : predicted protected attribute
- $g_\phi(\cdot)$: adversary network
- ϕ : adversary parameters

Adversary Loss

The adversary is trained using Binary Cross Entropy (BCE):

$$L_a = -\left[a \log(\hat{a}) + (1-a)\log(1-\hat{a})\right]$$

Symbols

- L_a : adversary loss
- a : protected attribute (e.g., gender or race)
- $\hat{a}$ (a-hat) : predicted protected attribute

This loss measures how well the adversary predicts the protected group.

Combined Objective

The goal of adversarial debiasing is to learn representations that are predictive of the target but uninformative about the protected attribute.

The combined loss is:

$$L_{total} = L_y - \lambda L_a$$

Symbols

- L_{total} : total training objective
- L_y : prediction loss (Binary Cross Entropy for the target)
- L_a : adversary loss (Binary Cross Entropy for the protected attribute)
- λ : fairness regularization weight that controls the trade-off between accuracy and fairness

Min–Max Optimization

Training becomes a two-player optimization problem.

Predictor objective

$$\min_\theta (L_y - \lambda L_a)$$

The predictor:

- minimizes the prediction loss L_y
- maximizes the adversary loss L_a

Adversary Objective

$$\max_\phi L_a$$

The adversary tries to correctly infer the protected attribute.

Final Training Objective

The adversarial training problem can be written as:

$$\min_\theta \max_\phi (L_y - \lambda L_a)$$

Google Colab project Link:
https://drive.google.com/drive/folders/1OewPA7dXknWxWaVxRNGH2Vb28WgOG4ZU?usp=sharing